

1. Contexte

Les tours de Hanoï sont un jeu de réflexion imaginé par le mathématicien français *Édouard Lucas*, et consistant à déplacer des disques de diamètres différents d'une tour de « départ » à une tour « d'arrivée » en passant par une tour « intermédiaire », et ceci en un minimum de coups, tout en respectant les règles suivantes :

- On ne peut déplacer plus d'un disque à la fois ;
- On ne peut placer un disque que sur un autre disque plus grand que lui ou sur un emplacement vide.

Fonctionnement (Exemple avec 3 disques) :

1. Déplacer le plus petit disque vers la tour d'arrivée.
2. Déplacer le disque de taille moyenne vers la tour intermédiaire.
3. Déplacer le plus petit disque sur le disque de taille moyenne.
4. Déplacer le disque le plus grand vers la tour d'arrivée.
5. Déplacer le plus petit disque vers la tour de départ.
6. Déplacer le disque de taille moyenne vers la tour d'arrivée.
7. Enfin, déplacer le plus petit disque sur la tour d'arrivée.

Pour n disques, le nombre minimum de coups nécessaires pour résoudre les tours de Hanoï est $2^n - 1$.

Dans le cadre de ce projet, on se placera dans un cadre simple, à savoir que nous n'avons que **3 tours** (que nous appellerons **pics**) et nous avons **3 disques** à déplacer, chacun de diamètre de 1, 2 et 3 (en cm par exemple). Nous référencerons les disques avec leur taille.

Dans l'environnement *Python* afin de symboliser la situation de jeu à un instant donné, vous pouvez créer une classe :

```
class Jeu_Hanoi:
    def __init__(self):
        self.pic = (np.zeros([3,3], dtype=np.int))
        self.nombre_palet = (np.zeros(3, dtype=np.int))
```

Cette classe contient :

a. Le **tableau « pic »** :

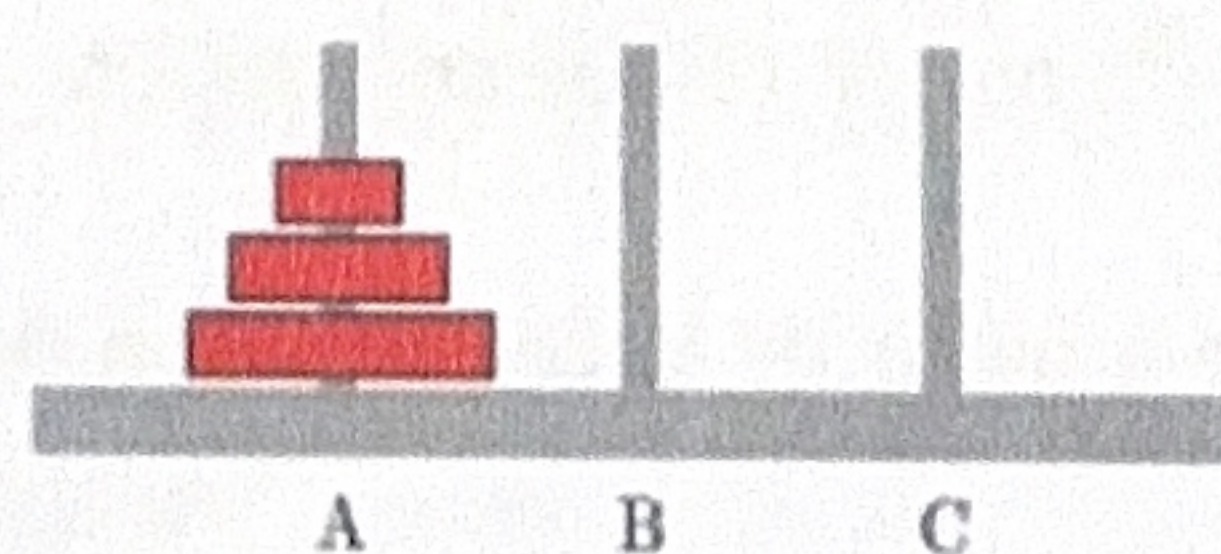
Le **tableau « pic »** est un **tableau Numpy** de dimensions 3×3 , représentant les trois pics sur lesquelles les disques peuvent être empilés.

- **Indices des lignes** : Le **premier indice** de ce tableau, c'est-à-dire l'**indice des lignes**, représente le **numéro du pic**. Les pics sont numérotés de 0 à 2 (gauche, intermédiaire, droite).
- **Indices des colonnes** : Le **second indice**, c'est-à-dire l'**indice des colonnes**, représente la **position des disques** empilés sur un pic donné. Par exemple, `pic[b,0]` contient le disque le plus bas (ou premier) sur le pic numéro b .
- **Valeur des cases** : Une case de ce tableau peut contenir soit un disque (représenté par un nombre entier correspondant au diamètre du disque), soit zéro si la case est vide.

Exemple :

Disque au milieu
Disque plus bas Disque plus haut

Pic 1	→	3	2	1
Pic_2	→	0	0	0
Pic_3	→	0	0	0



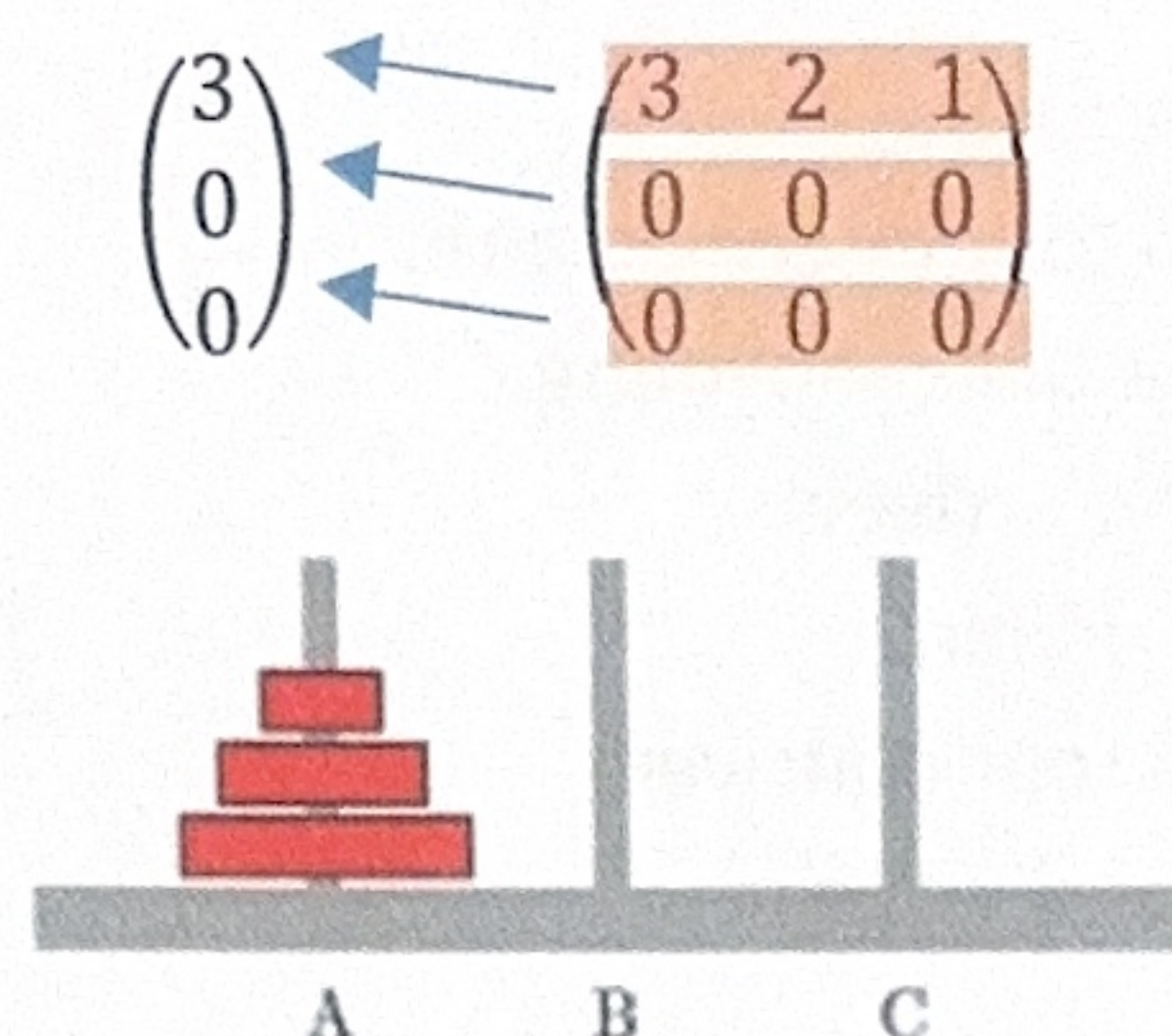
Le disque de diamètre 3 (le plus grand disque) est le plus bas du pic_1.

b. Le tableau « nombre_palet » :

Le tableau « nombre_palet » est un tableau Numpy de taille 3, où chaque élément représente le nombre de disques actuellement présents sur chacun des trois pics.

- **Indice des éléments** : Chaque indice du tableau correspond à un des trois pics. Par exemple, *nombre_palet[0]* donne le nombre de disques présents sur le pic numéro 0.
- **Valeur des cases** : Une case de ce tableau contient un entier qui indique combien de disques sont empilés sur le pic correspondant.

Exemple :

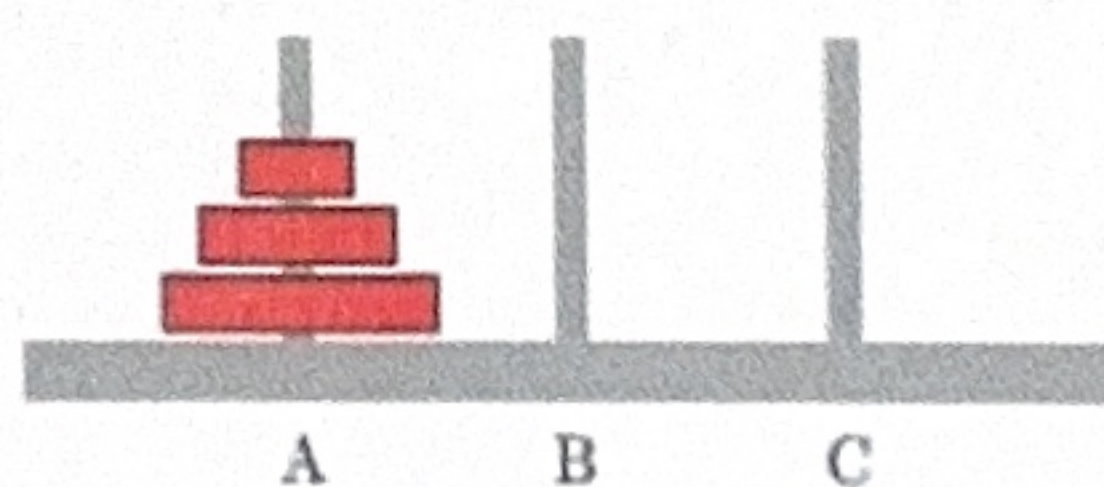


- `self.pic = np.zeros([3,3], dtype=np.int)` initialise le tableau « pic » avec des zéros. Cela signifie qu'au début du jeu, il n'y a aucun disque sur aucun pic (tous les emplacements sont vides).

- `self.nombre_palet = np.zeros(3, dtype=np.int)` initialise le tableau « nombre_palet » avec des zéros, ce qui signifie que le nombre de disques sur chaque pic est de 0 au début.

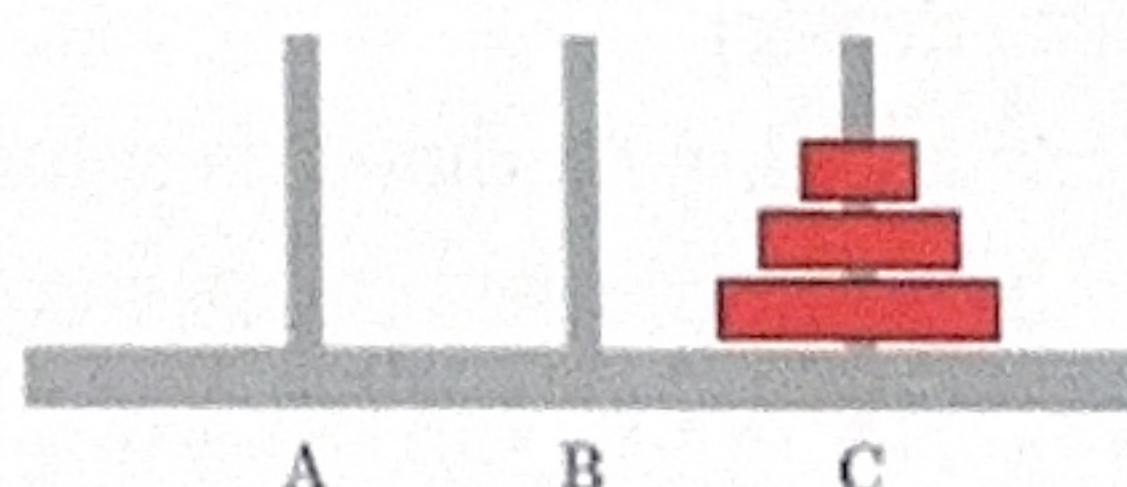
A l'initialisation la situation sera la suivante : tous les disques sont sur le premier pic rangés en taille décroissante en partant du bas donc 3 -> 2 -> 1 :

```
jeu = Jeu_Hanoi()
jeu.nombre_palet[0] = 3
jeu.pic[0,0] = 3
jeu.pic[0,1] = 2
jeu.pic[0,2] = 1
```



Et la situation finale désirée : tous les disques sont sur le dernier pic rangés en taille décroissante en partant du bas donc 3 -> 2 -> 1 :

```
Jeu_final = Jeu_Hanoi()
Jeu_final.nombre_palet[2] = 3
Jeu_final.pic[2,0] = 3
Jeu_final.pic[2,1] = 2
Jeu_final.pic[2,2] = 1
```



2. Base experte

Ce problème peut se résoudre algorithmiquement d'une manière optimale. Cependant nous allons mettre en place un système expert qui cherche une résolution à partir de règles simples.

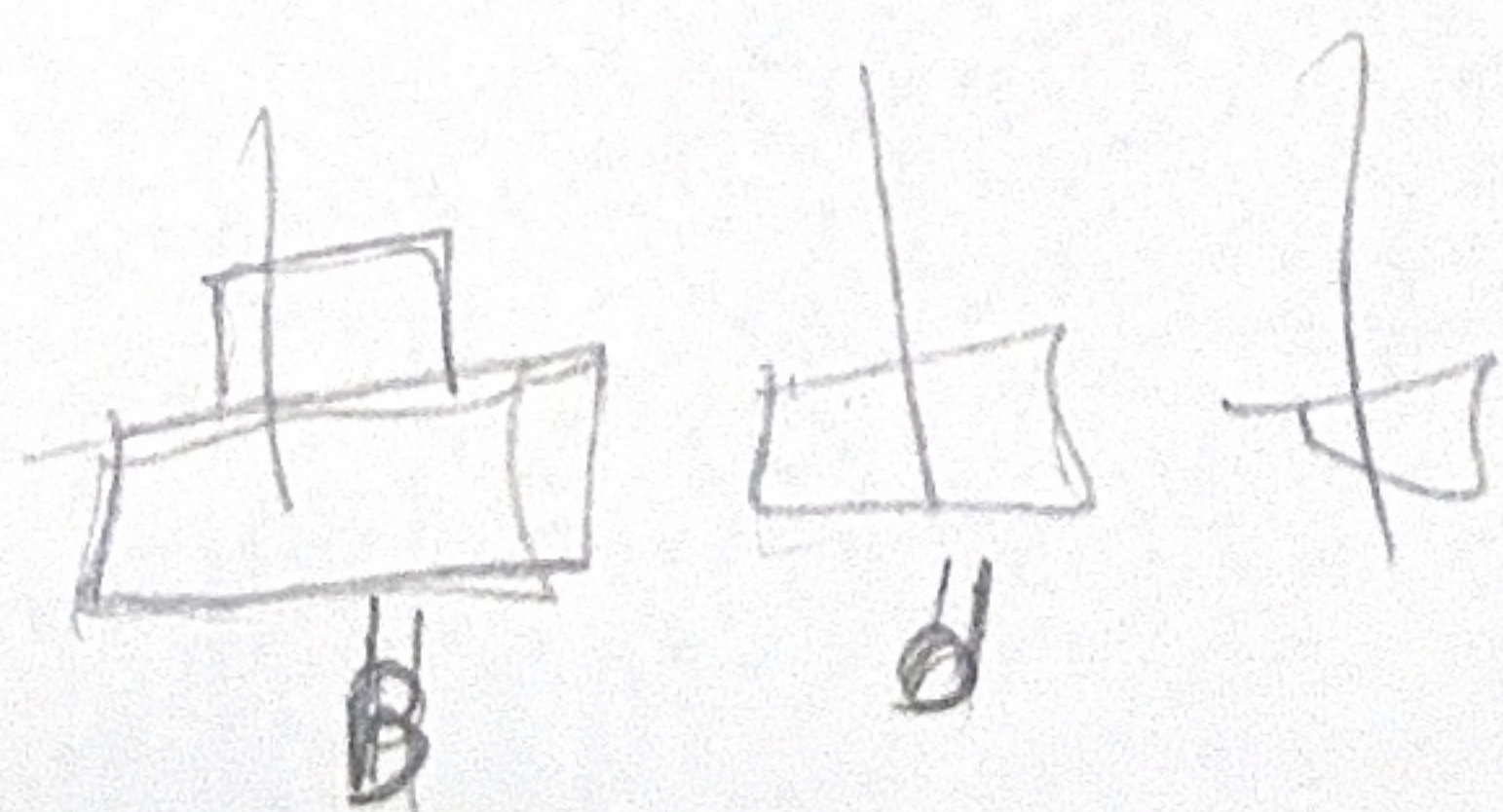
Après échange avec un expert, nous avons identifié les prémisses suivantes pour les règles qui vont constituer les éléments constitutifs de la base experte :

- On ne peut déplacer un disque d'un pic d'indice *b* sur un pic d'indice *d* que si le pic d'indice *b* contient au moins 1 disque.

Suggestion : cette condition peut être validée à partir de la valeur `jeu.nombre_palet[b] > 0`.

- On ne peut déplacer le disque supérieur du pic d'indice *b* (donc la case `jeu.pic[b, jeu.nombre_palet[b]-1]`) sur le pic d'indice *d* que si le pic d'indice *d* est vide (c-à-d `jeu.nombre_palet[d] = 0`) ou son disque en position supérieure est de taille plus grande (donc la case `jeu.pic[d, jeu.nombre_palet[d]-1]`).

Suggestion : cette condition peut être validée à partir de la comparaison de la valeur de `jeu.pic[b, jeu.nombre_palet[b]-1]` et la valeur de `jeu.pic[d, jeu.nombre_palet[d]-1]`



- Pour éviter de boucler sans fin durant la résolution du problème, on ne peut pas effectuer un déplacement si, après déplacement, on se retrouve dans une situation déjà étudiée.
- Suggestion : On veut passer de l'état initial à l'état final en effectuant le moins de déplacements possible.

Chaque disque est représenté par une **puissance de 2** : Disque 1 correspond à 2^0 , Disque 2 correspond à 2^1 et Disque 3 correspond à 2^2 . Chaque pic peut contenir des disques dans des positions spécifiques. Vous devez alors ajuster la représentation des disques sur chaque pic pour que chaque situation ait une valeur unique.

- Règle 1 : si (le pic 0 est non vide) et (le pic 1 est vide) :

- Règle 2 : si (le pic 1 est non vide) et (le déplacement du disque supérieur du pic 1 sur le pic 2 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 1 sur le pic 2).
- Règle 3 : si (le pic 2 est non vide) et (le déplacement du disque supérieur du pic 2 sur le pic 3 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 2 sur le pic 3).

- Règle 3 : si (le pic 2 est non vide) et (le déplacement du disque supérieur du pic 2 sur le pic 1 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 2 sur le pic 1).
- Règle 4 : si (le pic 1 est non vide) et (le déplacement du disque supérieur du pic 1 sur le pic 2 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 1 sur le pic 2).

- **Règle 4 :** si (le pic 1 est non vide) et (le déplacement du disque supérieur du pic 1 sur le pic 0 est autorisé) et (la situation résultante de ce déplacement n'a pas été observée) alors réaliser (le déplacement du disque supérieur du pic 1 sur le pic 0).
- **Métarègle :** En cas de plusieurs règles éligibles, on choisit la première.

- Métarègle : En cas de plusieurs règles éligibles, on choisit la règle de plus petit indice (si règle1 et règle3 sont applicable, on exécute la règle1).

3. Réalisation

Nous allons réaliser le programme permettant de résoudre le problème de la tour d'Hanoï à partir de la base experte définie par l'expert dans la section 2.

```
class Jeu Hanoi:
```

```
def __init__(self):
```

```
self.pic = (np.zeros([3,3], dtype=np.int))
```

```
self.nombre_palet = (np.zeros(3,dtype=np.int))
```

pass

```
import numpy as np # importe la bibliothèque NumPy, utilisée pour les calculs mathématiques et les vecteurs.
```

1. Réaliser une fonction `nombre_situation(jeu)` qui convertit une situation de jeu en un nombre unique et retourne ce nombre.

```
def nombre_situation(!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!).
```

```
##### # On commence par initialiser une variable 'nombre' à 0 # Elle servira à stocker le résultat final
```

```
# Première boucle : on parcourt les 3 pics (b = 0, 1, 2)
```

[illegible]

Deuxième boucle : on parcourt les palets présents sur le pic numéro b

'jeu.nombre_palet[b]' indique combien de palets sont posés sur ce pic

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1

Chaque palet a une position représentée dans 'jeu.pic[b, palet]'

On utilise une puissance de 2 (2^{**} ...) pour encoder cette position sous forme numérique

On ajoute aussi un décalage en fonction du numéro du pic ($b*3$)

Cela permet que deux palets identiques mais posés sur des pics différents donnent des nombres différents

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

On retourne le nombre unique qui représente toute la situation de jeu

return nombre

2. Réaliser une fonction `pic_vide(indice_pic, jeu)` qui vérifie que le pic d'indice `indice_pic` n'est pas sans palet, elle retourne un Boolean.

Fonction qui vérifie si un pic d'indice **indice** **pic** est vide ou non

```
def pic vide(||||||||||||||||||||||||||||||||||||||||):
```

```
reponse = !!!!!!!!!!!!!!!!!!!!!!!!!!!!! # 'jeu.nombre_palet[indice_pic]' = nombre de palets dans le pic choisi d'indice indice_pic, si ce nombre est >0, alors ce pic contient au moins un palet, s'il est égal à zéro, alors ce pic est vide.
```

```
return reponse # On retourne le résultat (True ou False)
```

3. Réaliser une fonction **regle_jeu**(indice_pic1, indice_pic2, jeu) qui vérifie si l'on peut déplacer le palet supérieur du pic d'indice indice_pic1 vers le pic d'indice indice_pic2, elle retourne un Boolean.
- # Fonction qui vérifie si l'on peut déplacer le palet supérieur du pic d'indice indice_pic1 vers le pic d'indice indice_pic2.
 # On peut faire ça si le pic d'indice indice_pic2 est vide, ou bien si le palet supérieur du pic d'indice indice_pic2 est de taille plus grande du palet supérieur du pic d'indice indice_pic1 qu'on voudrais le déplacer.
- ```
def regle_jeu(!!!!!!):
 if !!!!!: # si le pic d'indice indice_pic2 contient au moins un palet, il faut vérifier sa taille si elle est plus grande de celle du palet supérieur du pic d'indice indice_pic1.
 !!!!! # On récupère la taille du palet supérieur du pic d'indice indice_pic1 (pic de départ)
 !!!!! # On récupère la taille du palet supérieur du pic d'indice indice_pic2 (pic d'arrivée)
 !!!!! # reponse est vrai (on peut déplacer le palet supérieur du pic d'indice indice_pic1 vers le pic d'indice indice_pic2) si la taille du palet supérieur du pic d'indice indice_pic2 est plus grande de celle du palet supérieur de pic d'indice indice_pic1.
 else: # Pic d'arrivée est vide, on peut alors déplacer le palet supérieur de pic d'indice indice_pic1 vers le pic d'indice indice_pic2
 reponse = True

 return(reponse)
```
4. Réaliser une fonction **effectue\_deplacement**(indice\_pic1, indice\_pic2, jeu) qui effectue le déplacement du palet supérieur du pic d'indice indice\_pic1 vers l'indice indice\_pic2. La fonction déplace le palet et incrémente et décrémente le nbr de palets sur chacun des 2 pics, elle modifie donc la variable jeu.

```
def effectue_deplacement(!!!!!!):
 palet_depart = !!!!! # on détermine le palet qu'on va le déplacer, il se trouve dans la ligne associée au pic d'indice indice_pic1, et il est le palet supérieur
 !!!!! # on met zéro à la place de ce palet pour dire qu'il n'y a pas un palet dans cet endroit du pic de départ d'indice indice_pic1
 !!!!! # on décrémente le nombre des palets qui se trouvent dans le pic de départ d'indice indice_pic1
 !!!!! # on met le palet supérieur du pic d'indice indice_pic1 déjà stocké en palet_depart comme un palet supérieur du pic d'arrivée d'indice indice_pic2.
 !!!!! # on incrémente le nombre de palet associé au pic d'arrivée d'indice indice_pic2.
```

5. Réaliser une fonction **situation\_vue**(indice\_pic1, indice\_pic2, jeu, situation\_etudiee) qui, dans le cas d'un déplacement du palet supérieur du pic d'indice indice\_pic1 vers le pic d'indice indice\_pic2, vérifie que l'on ne retourne pas dans une situation déjà étudiée. La fonction retourne un Boolean.

```
def situation_vue(!!!!!!):
 # On souhaite faire le déplacement du palet supérieur du pic d'indice indice_pic1 vers le pic d'indice indice_pic2
 # La fonction doit vérifier si l'on ne retourne pas dans une situation déjà étudiée.
 # Attention on simule le déplacement donc on doit faire une copie temporaire de la situation stockée dans la variable jeu

 jeu_tmp = !!!!! # création d'un objet de classe Jeu_hanoi()
 # Faire une copie de la situation de l'objet jeu pris en paramètre de cette fonction
 !!!!!
 !!!!!

 !!!!! # effectuer le déplacement demandé en utilisant la fonction déjà définie effectue_deplacement.
 nombre_nouvelle_situation = !!!!! # récupère la nouvelle situation après le déplacement

 reponse = not(!!!!!!) # vérifie si la nouvelle situation est parmi les situations déjà étudiées et qui sont stockées dans la liste situation_etudiee prise en paramètre.
 # si la nouvelle situation après le déplacement ne trouve pas dans la liste des situations déjà étudiées -> return true
 # si c'est le cas alors on tombe sur une situation déjà étudiée -> return false !

 return reponse
```

À partir de ces différentes fonctions, vous allez développer le moteur d'inférence en chaînage avant qui à chaque itération, à partir de la situation initiale :

- sélectionne les règles éligibles suivant : si (**pic\_vide**(indice\_pic1, jeu)) et (**regle\_jeu**(indice\_pic1, indice\_pic2, jeu)) et (**situation\_vue**(indice\_pic1, indice\_pic2, situation\_etudiee)) alors règle éligible.
- applique la règle sélectionnée : effectue le déplacement « **effectue\_deplacement** », met à jour la liste « situation\_etudiee » et incrémente une variable comptant le nombre de coup joué.
- après chaque coup effectué, afficher le déplacement et la nouvelle situation de jeu.

---

```
def application_lregle(indice_pic1, indice_pic2, jeu, situation_etudiee):
Cette fonction va faire le déplacement de palet supérieur du pic d'indice indice_pic1 (pic de départ) vers le pic d'indice indice_pic2 (pic d'arrivée). Ce déplacement se fait
après qu'on teste si le pic de départ n'est pas vide (fonction pic_vide(indice_pic1, jeu)), et le pic d'arrivée est vide ou bien son palet supérieur est plus grand de celui du pic
de départ (fonction regle_jeu(indice_pic1, indice_pic2, jeu)) et que la situation après le déplacement n'est pas une situation déjà étudiée (fonction
situation_vue(indice_pic1, indice_pic2, jeu, situation_etudiee)).
if !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

#si ces conditions sont bien vérifiées, alors on affiche << « déplacement du palet » << l'identifiant de palet supérieur du pic de départ << « du pic » << indice de pic de départ
<< « vers le pic » << indice de pic d'arrivée.
print(!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!)

#On effectue maintenant le déplacement se fait par la fonction effectue_deplacement
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

#représentation de la situation de tour de Hanoi comme un nombre puissance de 2 (identifiant après l'application de ce déplacement)
nombre = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

#ajout l'identifiant de la situation de tour sur l'ensemble des situation déjà étudiées
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
return(True)

#le cas où le déplacement pas possible, ou bien le pic de départ est vide, ou bien le palet supérieur du pic d'arrivée est plus petit de la taille de celui du pic de départ, ou
bien la situation a déjà étudiées.
else:
return(False)
```

7. Cette inférence se fait jusqu'à obtenir la situation finale désirée.
- Tester votre programme et analyser le nombre de coup nécessaire pour résoudre le problème.
  - Dans le cas étudié, quel est le nombre de coups optimal pour résoudre le problème.
  - Peut-on ajouter d'autres règles pour optimiser la résolution ?
  - Selon le temps restant étendre votre programme à d'autres cas (par exemple, 4 ou 5 palets, 4 pyramides).
- 

```
On crée un objet de la classe Jeu_Hanoi() → c'est comme fabriquer un "plateau de jeu" vide
jeu = Jeu_Hanoi()

Initialisation de la situation de départ : Tous les disques sont dans le 1er pic (indice 0)
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

Liste qui va contenir toutes les situations déjà rencontrées (représentées par un tableau)
situation_etudiee = []

On convertit la situation initiale en un nombre unique et on l'ajoute dans la liste des situations étudiées
nombre = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
print(situation_etudiee)
#Mise en place des règles.
def test_regle(jeu, situation_etudiee):
 application = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
 rep = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! #règle 1 : déplacement de palet supérieur de pic 0 vers pic 1
```

Mise en place de la **situation finale** recherchée : tous les disques se trouvent dans le pic d'arrivée, trouver l'identifiant de cette situation à chercher, pour faire les comparaisons et savoir quand j'arrive à cette situation.

[illegible][illegible]

#Début du chainage avant  
Fin = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```
#nombre des coups initialisé à zéro
nbre coup = 0
```

```
Boucle qui tourne tant qu'on n'a pas atteint la situation finale
while not(Fin):
```

#application l'une des règles

```
application = !!!
```

### # Cas où une règle a été appliquée

if application:

!!!!!!!!!!!!!!!!!!!!!!!!!!!! # j'ajoute le nombre de coups

```
print("Situation à l'itération " + !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!) #numéro d'itération
```

```
print('!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!') # affichage de la situation
```

```
if !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! # Si la situation finale est atteinte → on arrête
```

```
print("Situation trouvée, fin de la partie")
```

```
Fin = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!# sortir de la boucle
```

```
else: #aucune règle applicable
```

```
print("Pb aucune règle applicable dans la situation actuelle")
```

```
print(jeu.pic)
```

*Fin* = !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

15 E

BAC E do BAC